INNOMATICS®
RESEARCH LABS

We don't just Train, We Transform Careers...!

ADVANCED CERTIFICATION IN

PROJECT BASED LEARNING

Curriculum

Online / Offline

WITH

DATA SCIENCE WITH GEN AI & AGENTIC AI

★ INTERNSHIP ★

JNTU-Hyderabad
No. 205, 2nd Floor, Fortune Signature, Near JNTU Metro Station, Kukatpally, Hyderabad, Telangana 500085
Contact Number: +91 99516 66670

HSR Layout-Bengaluru
No.576/B, 3rd FLOOR, Swetha Arcade, Service Road, 100 FT ROAD, 6TH SECTOR, Hsr Layout, Bengaluru, Karnataka 560068
Contact Number: +91 99516 66672

Dilshuknagar-Hyderabad
2nd Floor, Beside Victoria Memorial Metro Station, Pillar No.1623 Narsimhapuri Colony Arch, Kothapet, Hyderabad, Telangana 500035
Contact Number: +91 99516 66674

Kothrud-Pune
2nd Floor, Sai Sayaji Apartment, 201 Paud Rd, beside VANAZ Metro, above Bata showroom, Ramkrishna Paramhans Nagar, Kothrud, Pune, Maharashtra 411038
Contact Number: +91 99516 66671

Table of Contents

Page No

Our Awards & Collaborations 03

Course Objective 04

Key Highlights Of The Training 05

Tools Covered In The Course 06

Introduction & Walk Through The Course 07

Module 1 - Python Programming Foundations & Backend Engineering 08

Module 2 - AI Engineering : Machine Learning & Neural Networks 12

Module 3 - No-Code Workflow Automation using n8n 18

Module 4 - Generative AI Foundations: NLP, Transformers & LLMs 20

Module 5 - Building Document Intelligence Pipelines 24

Module 6 - Building Intelligent Retrieval Systems: RAG, Hybrid Search & Evaluation 26

Module 7 - Agentic Systems Engineering with LangGraph & MCP 29

OUR AWARDS & COLLABORATION

TIH Foundation for IoT & IoE

TIH at IIT Bombay

TIH at IIT Patna

futureskills prime

NASSCOM

N·S·D·C National Skill Development Corporation









200+ Hackathons	**LMS** Lifetime Access	**20,000+** Career Transitions	**100%** Placement Assistance	**650+** Hiring Partners
600+ Batches	**500+** Webinars	**4 Lakh+** Hours Delivered	**500+** Industry Experts	**Capstone** Projects

- 50+ Clients for Corporate Trainings
- 100+ College MOUs Signed

COURSE OBJECTIVE

1. Engineer Python applications using core data structures, functions, classes, file handling and exception handling, and deploy them as containerized FastAPI REST services with Pydantic validation and Git version control, to solve real-world text and data-processing problems.

2. Design, tune and validate classification and regression pipelines in scikit-learn on tabular and text data, using cross-validated search and metrics such as precision, recall, F1, RMSE and R^2, to select the best-performing model for a business problem.

3. Analyze how BOW and TF-IDF, Word2Vec, RNN, LSTM/GRU and Transformer models each overcome the limits of the one before, and apply pretrained HuggingFace models with zero-shot, few-shot and chain-of-thought prompting to solve core NLP tasks.

4. Develop LCEL chains from chat models, prompt templates and Pydantic output parsers, and instrument them with LangSmith or Langfuse tracing, to deliver observable document-processing pipelines.

5. Construct RAG pipelines in LangChain using document loaders, chunking, embeddings, vector stores, metadata filtering and hybrid retrieval, and benchmark them with LLM-as-a-judge and RAGAS metrics, to optimize answer accuracy and faithfulness.

6. Engineer tool-using, supervisor-based multi-agent systems in LangChain and LangGraph with memory, middleware, human approval and MCP tools, and validate them with LangSmith evaluators and Agent Chat UI, to ensure reliable tool use and agent handoffs.

KEY HIGHLIGHTS OF THE TRAINING

Total Duration: 6 Months
3 Months Training + 3 Months Internship

Instructor-Led LIVE Sessions
Both Online & Offline

Life-time LMS Access
All the course related content

LIVE Doubt Resolution
One-On-One Mentoring session

Topic-wise Assignments
All the Modules

Weekly Quizzes
Evaluate comprehension and application of key concepts, providing timely feedback to enhance learning outcomes.

Job Fitment Assessment
Mastery of key concepts and practical skills, ensuring readiness for real-world challenges

20+ Use Cases
Providing real-world context for learning

Vibrant Hackathons
Challenging students to solve real-world problems through data-driven solutions

Mock Interviews
Helping students refine their communication and technical skills for professional success

Discord Community Access
Enthusiasts collaborate, share insights, and network with industry professionals

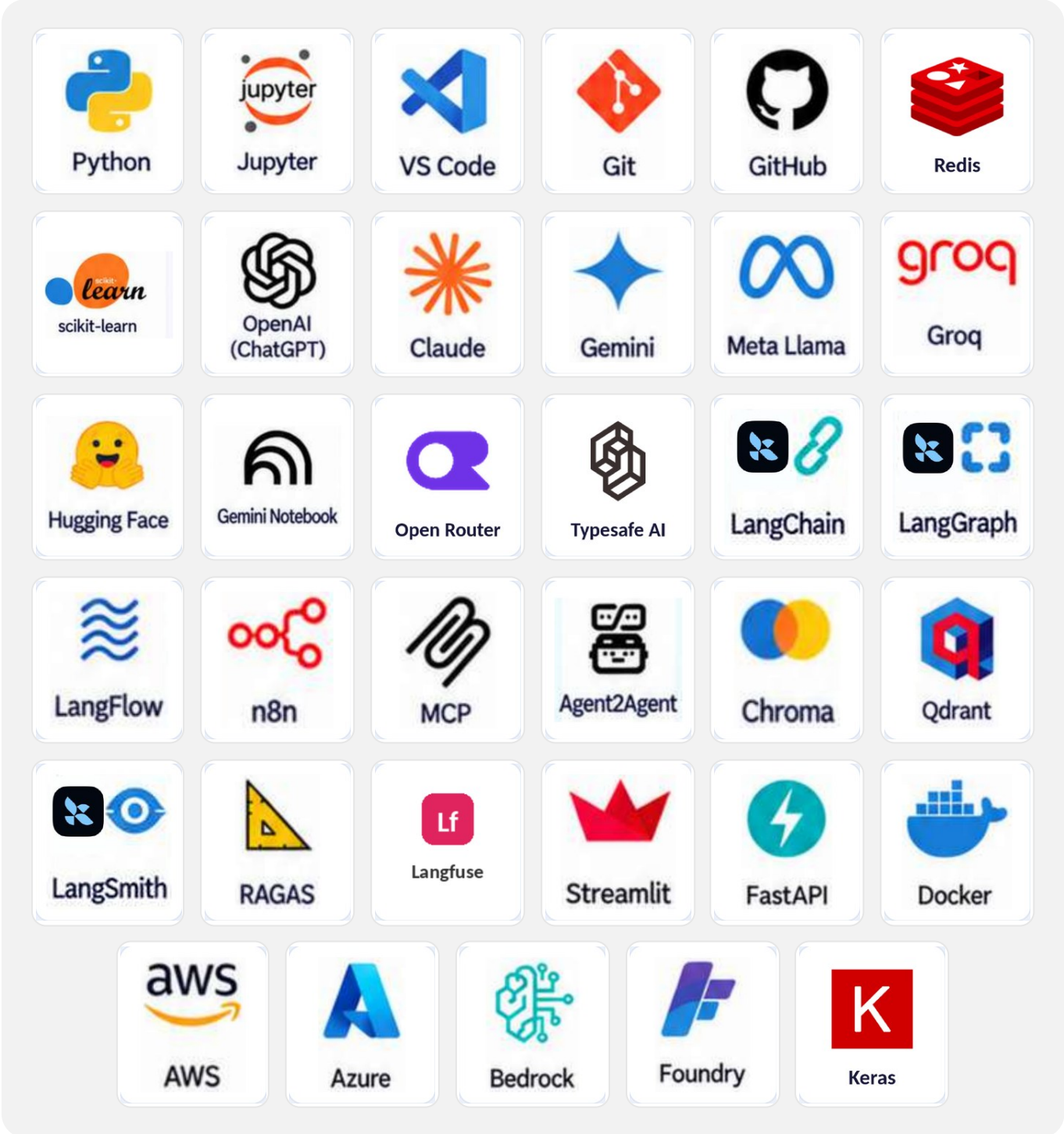
Guaranteed Internship
Extensive in-house 2 to 3 months internship. Gain valuable hands-on experience in real-world settings.

Dedicated Soft Skills
Equipping you with essential communication, teamwork, and leadership abilities for success

Resume Tailoring
Techniques to highlight your skills and experience, increasing your chances of securing job opportunities

100% Job Assistance
Supported by our extensive network of hiring partners and personalized career guidance

TOOLS COVERED IN THE COURSE



Note: Some of the tools will be covered as part of the internship program.*

INTRODUCTION & WALK THROUGH THE COURSE

This programme takes you from Python fundamentals to building, evaluating and deploying agentic AI systems. The curriculum is organised into seven modules, each ending in hands-on projects that feed your project portfolio.

1	**Python Programming Foundations & Backend Engineering**	1 PROJECT
2	**AI Engineering : Machine Learning & Neural Networks** Libraries: sklearn, nltk, tensorflow, keras, optuna	2 PROJECTS
3	**No-Code Workflow Automation using n8n**	1 PROJECT
4	**Generative AI Foundations: NLP, Transformers & LLMs** Libraries: TENSORFLOW, KERAS, GENSIM	3 PROJECTS
5	**Building Document Intelligence Pipelines** Libraries: HuggingFace, Google Gemini API and Groq API, Langchain	1 PROJECT
6	**Building Intelligent Retrieval Systems: RAG, Hybrid Search & Evaluation** Libraries: Langchain, LangGraph, LangSmith, RAGAS	2 PROJECTS
7	**Agentic Systems Engineering with LangGraph & MCP** Libraries: Langchain, LangGraph, LangSmith	3 PROJECTS

MODULE 1

01

PYTHON PROGRAMMING FOUNDATIONS & BACKEND ENGINEERING

1 PROJECT

1. Introduction to Python

- What is Python and why it is used for data science and AI
- Install Anaconda and launch Jupyter Notebook
- Notebook basics: code and markdown cells, running cells, shortcuts
- Python syntax: indentation, comments, case sensitivity

2. Datatypes and Variables

- Identifiers, naming rules and keywords
- Variables and core datatypes: int, float, str, bool, None
- type() and type casting

3. Operators and Conditional Statements

- Arithmetic, assignment, comparison, logical and identity operators
- If, if-else, if-elif-else, nested conditions
- Truthy and falsy values

4. While Loops

- What a loop is and why we use one
- while syntax and the loop condition
- Break, continue and infinite loops
- Simple pattern problems

5. Data Structures I: Lists, Tuples and Sets

- Lists: creating, indexing, slicing, nested lists
- List methods: append, insert, remove, pop, sort, index, count
- Mutability: why lists change and tuples do not
- Tuples: creation, indexing, unpacking; lists vs tuples
- Sets: uniqueness and methods (add, remove, discard)

6. Data Structures II: Dictionaries and Strings

- Dictionaries: key-value pairs, add, update, delete
- Access with [] and get(); keys(), values(), items(); nested dictionaries
- Sets vs dictionaries; mutable vs immutable types (recap)
- Strings: indexing, slicing, immutability
- Common string methods: lower, upper, strip, split, join, replace, find, startswith
- F-strings and format(); chr() and ord() (quick look)

7. For Loops

- for with range(), lists, strings and dictionaries
- enumerate and zip
- Break, continue and nested loops
- List and dictionary comprehension

8. Functions

- What a function is and why we use one
- Def, parameters, arguments, calling a function
- Positional, keyword and default arguments
- return vs print
- *args and **kwargs
- Scope: local vs global variables
- Lambda functions
- Map, filter and sorted with a key
- Recap: functions can be passed around and returned
- Writing a simple decorator
- @ syntax and functools.wraps
- Examples: timer, logger, retry

9. Object-Oriented Programming

- Why classes: bundling data and behavior
- Defining a class; creating objects; __init__ and self
- Attributes and methods (instance vs class variables)
- Inheritance: single inheritance, super(), method overriding
- Polymorphism in plain words (the same method name behaves differently per class)
- Encapsulation: _protected and __private naming, getters and setters with @property
- Abstraction: Abstract class, Abstract Method

10. Text and JSON file Handling

- Read and write text files with with open(); check a file exists
- Read and write JSON with the json module

11. Exception Handling

- What an exception is; reading a traceback
- Common exceptions: ValueError, TypeError, KeyError, IndexError, FileNotFoundError
- Try, except, else, finally; catching specific exceptions
- Raise and custom exception classes

12. Request-Response Model and REST Fundamentals

- Client-server and the request-response model
- HTTP methods: GET, POST, PUT, DELETE
- Status codes (200, 201, 400, 401, 404, 500), headers and JSON payloads
- What REST is, in simple terms

13. Type Annotations and Pydantic

- Why type hints (readability, fewer bugs, editor help)
- Hints for variables, parameters and return values
- List[str], dict[str, int], Optional, Union (or |)
- Pydantic models and field types

14. FastAPI I: Basics and Auto Docs

- Why FastAPI (speed, type hints, automatic docs)
- Install, create an app, run with uvicorn (including --reload)
- Routes and the HTTP methods (@app.get, @app.post)
- Path parameters and query parameters
- Automatic docs at /docs (Swagger UI) and /redoc

15. FastAPI II: Pydantic Models and CRUD

- Request body models for POST and PUT
- Response_model and what validation errors look like
- Status codes and HTTPException
- Hands-on: a full to-do API with Pydantic request and response models, tested from /docs

16. FastAPI III: Async Execution

- Sync vs async: def vs async def, await, and the event loop in plain words
- How FastAPI runs def endpoints in a thread pool and async def endpoints on the event loop
- Hands-on: a slow endpoint with time.sleep vs asyncio.sleep, tested with several simultaneous requests

17. FastAPI IV: Project Structure and Configuration

- Python modules and packages (import, __init__.py)
- Typical layout: main.py, routers/, models/, services/
- Environment variables, .env files and python-dotenv
- A settings class with pydantic-settings

18. FastAPI V: Caching

- Why cache?
- What to cache and what not to cache?
- Types of caches (browser, CDN, reverse proxy, in-memory, Redis, database)
- Cache terminology
- In-memory caching with `functools.lru_cache`
- Shared caching with Redis using `redis-py`
- Common pitfalls

19. Introduction to Unix: Navigation and Files

- What Unix, Linux and the terminal are, and why they matter for AI work
- Navigation: pwd, ls, cd, mkdir, touch, relative vs absolute paths
- Files: cp, mv, rm, cat, head, tail, less, wc
- Help and shortcuts: man, --help, tab completion, history

20. Introduction to Git: Local Version Control

- What version control is and why we use it
- Install and configure Git (name, email)
- git init, status, add, commit, log, diff
- Writing good commit messages
- What must never be committed; .gitignore
- Undoing changes: git restore, git revert (basic)

21. Containerization with Docker I: Images and Containers

- The problem: "it works on my machine"
- Images vs containers, and Docker vs a virtual machine
- Install Docker Desktop and check the setup
- Docker Hub and pulling images
- docker run, ps, stop, rm, logs, exec; port mapping

22. Docker II: Dockerfile and Compose

- Write a Dockerfile for a FastAPI app (FROM, WORKDIR, COPY, RUN, CMD)
- docker build, tagging and running with port mapping
- Passing environment variables (--env-file); keeping secrets out of the image
- .dockerignore
- Volumes and docker-compose (basic demo)

MODULE PROJECT

★ MODULE PROJECT

Building and Deploying a Python Backend Service

1. Understand a real-world problem that needs a data-driven service, and define the data it manages, the operations users need and how success will be measured.
2. Set up the development environment with Anaconda and Jupyter Notebook, and prepare a project folder from the Unix terminal.
3. Model the problem's data using core data types, variables, lists, tuples, sets, dictionaries and strings, and decide which structure fits each kind of data.
4. Write the core logic with operators, conditional statements, while and for loops, and comprehensions to validate, filter and transform the data.
5. Organize the logic into reusable functions, using default arguments, *args and **kwargs, lambda functions, and map, filter and sorted.
6. Add decorators (such as a timer, a logger and a retry decorator) so common behavior is applied without repeating code.
7. Redesign the components as classes with encapsulation, inheritance, polymorphism and abstract base classes, so different variants of the same idea share one interface.
8. Store and load data with text and JSON files, checking that files exist before reading them.
9. Make the code robust with exception handling, catching specific errors, raising custom exceptions, and giving clear error messages.
10. Add type annotations throughout and define Pydantic models, so data is checked automatically.
11. Design the service as a REST API, choosing HTTP methods, status codes and JSON payloads for each operation.
12. Build the API with FastAPI, using path and query parameters, request and response models, HTTPException and the automatic docs at /docs and /redoc, and implement full create, read, update and delete operations.
13. Use async endpoints for operations that wait on slow work, and check that several simultaneous requests do not block each other.
14. Organize the code into a project structure with routers, models and services, and move secrets and settings into environment variables, a .env file and a settings class.
15. Speed up repeated work with caching, using functools.lru_cache for in-process results and Redis for shared results, and decide what should not be cached.
16. Track the project with Git using clear commits and a .gitignore that keeps secrets out.
17. Package the service with a Dockerfile, build and run the container with port mapping and environment variables, and use Docker Compose to run it with supporting services such as Redis.

OUTCOME | **Deliver an end-to-end, tested and containerized API service that runs the same way on any machine, with documented endpoints and decisions.**

MODULE 2

02

AI ENGINEERING : MACHINE LEARNING & NEURAL NETWORKS

2 PROJECTS

MODULES / LIBRARIES COVERED DURING THIS MODULE

sklearn | nltk | tensorflow | keras | optuna

1. Introduction to Machine Learning

- Why machine learning?
- AI vs ML vs Deep Learning vs LLMs
- Supervised vs unsupervised learning
- Regression vs classification
- The ML lifecycle: problem, data, preprocessing, model, evaluation, tuning

2. Problem Definition and Objectives

- From business problem to ML problem
- Identifying features and target
- Deciding regression or classification
- Activity: convert 5 business problem statements into ML problems

3. Data Collection and Exploratory Data Analysis

- Data sources and loading datasets with pandas
- Understanding the data: head, info, describe, shape
- Univariate, bivariate and multivariate analysis with seaborn
- Reading charts: distributions and relationships

4. Data Quality and Train-Test Split

- Identifying and removing duplicates
- Handling missing values
- Detecting outliers
- Train_test_split: test_size, random_state, stratify
- Data leakage: what it is and the rule 'fit on train, transform on test'

5. Data Transformations: Scaling and Encoding

- Why scaling matters?
- StandardScaler and MinMaxScaler
- Nominal vs ordinal features: OrdinalEncoder and OneHotEncoder
- Using Column transformer in Scikit-learn

6. Feature Engineering and Text Features

- Feature transformation and creating new features
- Feature selection and feature importance
- Text cleaning with nltk: lowercase, tokenization, stop words, stemming
- Bag of Words and TF-IDF with CountVectorizer and TfidfVectorizer
- Cosine similarity between two documents

7. Mathematical Foundations

- Introduction and why Linear Algebra
- Fundamentals of Vectors and Matrices
- Linear Transformation
- Dot Product of vectors
- Angle between two vectors
- Projection of a vector onto another vector
- Distances metrics: Euclidean, Manhattan, Minkowski, Cosine and Jaccard Similarities

8. K-Nearest Neighbors

- What classification is?
- The scikit-learn pattern: fit, predict, score
- KNN idea: similar items vote; distance and the choice of k
- Effect of feature scaling on KNN
- Hands-on: train KNN and change k

9. Decision Trees

- How a tree asks questions to split data (intuition)
- Max_depth and overfitting
- Visualizing a tree with plot_tree and reading feature importances
- DecisionTreeRegressor: what changes for regression
- Hands-on: classification and regression trees

10. Linear Regression

- What regression is and how it differs from classification
- Linear regression: line of best fit, coefficients and intercept (intuition)
- Predicting with LinearRegression in scikit-learn
- Hands-on: house-price style regression with three models

11. Regression Evaluation Metrics

- MAE, MSE and RMSE: meaning and units
- R-squared and adjusted R-squared
- Residual plots: what a good fit looks like
- Comparing models with the right metric

12. Logistic Regression

- From a line to a probability: the sigmoid idea
- Decision threshold
- Reading coefficients: which features push toward which class
- Sentiment classification with TF-IDF and logistic regression
- Logistic regression as the simplest neuron

13. Classification Evaluation Metrics

- Why accuracy can mislead (imbalanced data)
- Confusion matrix: TP, FP, TN, FN
- Precision, recall and F1 score
- Classification_report in scikit-learn
- ROC curve and AUC (intuition only)
- Choosing a metric for the problem (spam, medical, fraud)

14. Overfitting, Underfitting and the Bias-Variance Tradeoff

- Underfitting vs overfitting using simple plots
- Bias and variance in plain words
- Learning curves and train vs validation scores in scikit-learn
- Ways to reduce overfitting: simpler models, more data, regularization
- Regularization preview: Ridge and Lasso as settings that tame a model

15. Introduction to Ensembles: Bagging and Random Forest

- Why combine models: the wisdom-of-the-crowd idea
- Voting ensembles (VotingClassifier)
- Bagging: many models on different samples
- Random Forest: many trees with random features
- Out-of-bag score and feature importance
- Hands-on: compare a single tree with a Random Forest

16. Boosting and Model Comparison

- Boosting: models that learn from earlier mistakes
- AdaBoost and Gradient Boosting (intuition)
- GradientBoosting in scikit-learn, and XGBoost
- Bagging vs boosting: when to use which
- Hands-on: compare KNN, logistic regression, tree, Random Forest and boosting on the same data

17. Model Building and ML Pipelines

- Why doing steps by hand causes repeated code and leakage
- Pipeline: chaining preprocessing and a model
- ColumnTransformer: separate steps for numeric, categorical and text columns
- Text pipeline: TfidfVectorizer followed by a classifier
- Saving and loading a model with joblib

18. Cross-Validation and Hyperparameter Tuning

- Parameters vs hyperparameters
- Cross-validation: why one split is not enough
- GridSearchCV
- RandomizedSearchCV
- Model selection: choosing the best model without overfitting the test set

19. Unsupervised Learning I: Introduction and K-Means

- Supervised vs unsupervised learning (recap) and where clustering is used
- What clustering means: grouping without labels
- K-Means idea: pick centers, assign points, update, repeat
- Why scaling matters for K-Means
- Hands-on: K-Means in scikit-learn on a 2D dataset with cluster plots

20. Unsupervised Learning II: Choosing K, Segmentation and Text Clustering

- Choosing the number of clusters: elbow method and silhouette score
- Customer segmentation: interpreting each segment
- Clustering documents with TF-IDF and reading the top words per cluster
- Visualizing clusters in 2D with Tsne

21. Introduction to Neural Networks

- Why Neural Networks?
- Analogy between biological and artificial neuron
- Single layer perceptron model
- Understanding weights, bias and activation

22. Perceptron Model and Activation Functions

- Perceptron learning rule
- Artificial neuron
- MLP/FCNN/ANN model and nomenclature of ANN
- Intuition for activation functions: linear, sigmoid, tanh, ReLU (Leaky ReLU, PReLU) and softmax

23. Training Neural Network and Optimizers

- Forward pass with formulation
- Backward pass
- Understanding the backpropagation algorithm (using AutoGrad methods like GradientTape in TensorFlow)
- Intuition for optimizers: gradient descent, mini-batch, momentum, RMSProp and Adam

24. ANN Model Building

- Classification model building using Keras
- Regression model building using Keras
- Evaluating the model

25. Overfitting in ANN

- L1 and L2 regularization
- Dropout regularizer
- Early stopping
- Batch normalization

26. Hyperparameter Tuning in ANN

- Introduction to Optuna
- Creating a study in Optuna
- Finding hyperparameters using Optuna
- Hyperparameter importance using Optuna

MODULE PROJECTS

★ PROJECT 1 OF 2

End-to-End Machine Learning and Neural Networks

1. Understand a real-world business problem, convert it into a machine learning problem, and define the features, the target, the task type (classification or regression) and the success measure.
2. Load data from files, databases or public sources with pandas, and inspect its structure, types and summary statistics.
3. Explore the data with univariate, bivariate and multivariate analysis, and note the distributions, relationships and class balance that will shape the modeling.
4. Clean the data by removing duplicates, handling missing values and detecting outliers, then split it into training and test sets with a leakage-free approach.
5. Transform the data with scaling and encoding inside a column transformer, and engineer and select features, including text features such as Bag of Words and TF-IDF where the data contains text.
6. Build baseline models with K-Nearest Neighbors, Decision Trees, Linear Regression or Logistic Regression, using distance and similarity measures where they apply.
7. Evaluate the baselines with metrics that match the task, such as MAE, RMSE and R^2 for regression, or precision, recall, F1 and ROC-AUC for classification.
8. Diagnose overfitting and underfitting with learning curves, and reduce them with simpler models, more data and regularization.
9. Improve performance with ensembles (bagging, Random Forest and boosting), and compare all models on the same data.
10. Combine preprocessing and the chosen model into a single pipeline, and tune its hyperparameters with cross-validation and grid or randomized search.
11. Optionally explore the data without labels, using K-Means (choosing the number of clusters) or document clustering, to find segments or topics.
12. Build a neural network with Keras for the same problem, using suitable activation functions, loss and optimizer, and compare it with the best classical model.
13. Control overfitting in the network with L1/L2 regularization, dropout, early stopping and batch normalization, and tune its hyperparameters with a tool such as Optuna.
14. Select the best model on validation evidence, evaluate it once on the test set, and save it with its pipeline, for example with joblib or the Keras format.
15. Deploy the saved model as a simple prediction application, such as a web app or API, that takes new inputs and returns predictions.

OUTCOME | **Deliver an end-to-end machine learning solution that runs from raw data to deployed predictions, with documented decisions and results.**

★ PROJECT 2 OF 2

Unsupervised Machine Learning (Clustering and Segmentation)

1. Understand a real-world business problem where no labels exist, such as grouping customers or documents, and define what a useful group would look like.
2. Load data from files, databases or public sources with pandas, and inspect its structure, data types and summary statistics.
3. Explore the data with univariate, bivariate and multivariate analysis, and look for natural groupings, skewed features and correlated features.
4. Clean the data by removing duplicates, handling missing values and detecting outliers that could distort the clusters.
5. Prepare the features by scaling numeric columns and encoding categorical ones inside a column transformer, and convert text into numeric features with TF-IDF when the data is text.
6. Choose a suitable distance or similarity measure, such as Euclidean distance or cosine similarity, based on the type of data.
7. Build a baseline K-Means model and see how its clusters form and move.
8. Choose the number of clusters with the elbow method and the silhouette score, and check that the result is stable across runs.
9. Interpret each cluster by profiling its average feature values, or for documents by reading the top words, and give each cluster a business meaning and a name.
10. Visualize the clusters in two dimensions with a method such as t-SNE, and check that they are well separated.
11. Combine preprocessing and clustering into one pipeline so that new data is transformed the same way.
12. Save the pipeline, for example with joblib.
13. Deploy it as a simple application, such as a web app or API, that takes a new record or document and returns its cluster and description.

OUTCOME | **Deliver an end-to-end clustering solution with named segments, visual evidence and actions the business can take.**

MODULE 3

03 NO-CODE WORKFLOW AUTOMATION USING N8N

1 PROJECT

1. n8n fundamentals and first automation

- Automation basics
- N8n overview and editor
- Triggers and data flow
- Connecting apps and APIs: the HTTP Request node with public APIs, and Google Sheets and Gmail.
- Hands-on: a scheduled daily digest workflow.

2. Logic, data and error handling

- Routing: IF and Switch nodes.
- Data handling: Split Out, Loop Over Items, Aggregate, Edit Fields, and a simple Code node.
- Loops and batches: processing many records with Loop Over Items.
- Basic reliability: reading execution logs, debugging, Retry On Fail and Continue On Fail.
- Hands-on: a form or webhook intake that validates and cleans entries, routes them by category, logs them to a sheet and emails a confirmation.

3. API-Based Data Collection and Processing

- Form and Webhook triggers
- HTTP Request and API authentication
- Asynchronous job polling
- Data transformation and mapping
- Deduplication and validation

4. Scheduled Feed Collection and Text Processing

- RSS and feed integration
- Scheduled workflows
- Text cleaning and preprocessing
- Regular expressions
- Filtering and record selection

5. File Upload and Media Processing

- Webhook-based file uploads
- Binary data handling
- File-type and MIME validation
- Command-line media processing
- JSON response generation

6. Workflow Version Control and Deployment

- Workflow API integration
- GitHub or repository API integration
- File comparison and change detection
- Environment-based routing
- Automated deployment workflows

MODULE PROJECT

★ MODULE PROJECT

No-Code Workflow Automation with n8n

1. Understand a real-world manual process and define what the workflow must automate, its inputs, its outputs and how success will be measured.
2. Set up n8n and connect the apps involved, such as Google Sheets and Gmail, with secure credentials.
3. Build the first workflow with a trigger (manual, schedule, form or webhook) and trace how data moves between nodes.
4. Add logic with IF and Switch nodes, and handle records with Edit Fields, Split Out, Loop Over Items, Aggregate and a simple Code node.
5. Choose one or more options to build the core of the workflow:
   - A **API-based data collection and processing.**
     Collect data with HTTP Request nodes and API authentication, poll asynchronous jobs until they finish, map the data to one structure, and remove duplicates and invalid records.
   - B **Scheduled feed collection and text processing.**
     Collect RSS or feed content on a schedule, clean the text with Edit Fields, the Code node and regular expressions, and filter the records you need.
   - C **File upload and media processing.**
     Accept files through a webhook, validate the file type and MIME type, process the media with a command-line tool such as FFmpeg on a self-hosted n8n, and return a JSON response with the result.
6. Log results to a destination such as Google Sheets and send notifications such as a confirmation email or a summary.
7. Add reliability with Retry On Fail and Continue On Fail, and use the execution logs to debug failures.
8. Test the workflow with realistic and faulty inputs, and fix the problems you find.
9. Store the workflow in a repository such as GitHub through its API, detect changes by comparing versions, and commit only when something has changed.
10. Deploy the approved workflow to the chosen environment (development, staging or production) through the n8n API, keeping secrets in credentials and out of the stored files.

OUTCOME | **Deliver an automated, tested and version-controlled workflow that runs without manual work in real-world use.**

MODULE 4

04 GENERATIVE AI FOUNDATIONS: NLP, TRANSFORMERS & LLMS

3 PROJECTS

MODULES / LIBRARIES COVERED DURING THIS MODULE

TENSORFLOW | KERAS | GENSIM

1. Introduction to Generative AI

- Intro to Generative AI
- BOW, TF-IDF
- Issues with BOW, TFIDF

2. Word Embeddings - Word2Vec

- Word2Vec
- How Word2Vec algorithm works (CBOW, Skip-gram)
- Issues with Word2Vec: one fixed vector per word

3. Sequence modelling with RNN

- Intro to RNN
- Types of RNN (one-to-many, many-to-one, many-to-many)
- Issues with RNN

4. LSTM, GRU

- Limitations of RNN
- Idea of LSTM (cell state, forget/input/output gates) and GRU (update/reset gates) - intuition only.
- Issues with LSTM/GRU

5. Transformer Architecture I

- Why Transformers (parallelism, long-range dependencies)
- Positional Encoding
- Self and Multihead Attention
- Masked Multihead Attention

6. Transformer Architecture II

- GPT with Autoregressive Language Modelling
- BERT with Auto-encoding Language Modelling
- Encoder-only vs decoder-only vs encoder-decoder models (BERT, GPT, T5)

7. HuggingFace API

- Introduction to HuggingFace
- HuggingFace Embeddings
- Case Study: Sentiment Analysis, Named Entity Recognition, Parts of Speech Tagging and Question and Answering

8. NLP Tasks using AutoClasses

- Auto classes - Config, Tokenization and Models

9. Text Generation & Question Answering

- Text Summarization
- Text Translation

10. Evaluation Metrics

- Introduction evaluation Metrics
- BLEU, ROUGE
- METEOR, Perplexity

11. HuggingFace for Image and Audio data

- Image classification
- Image detection
- Audio Classification

12. Large Language Models

- What is an LLM? scale
- Popular LLM families: open vs closed models
- Capabilities: text generation, summarization, translation, Q&A, reasoning, code generation, information extraction, classification, multimodal inputs
- Limitations: hallucinations, outdated knowledge, bias, context window limits, cost

13. Prompt Engineering I

- What is a prompt, system vs user prompts
- Temperature, top-p
- Good practices

14. Prompt Engineering II

- Prompting techniques: zero-shot, few-shot, chain-of-thought, self-consistency, Tree of Thought

MODULE PROJECTS

★ PROJECT 1 OF 3

Text Classification, from Word Counts to Sequence Models

1. Understand a real-world text classification problem, such as sentiment, spam or ticket routing, and define the labels and the success measure.
2. Clean and prepare the text data by lowercasing, removing noise and tokenizing it.
3. Build a baseline classifier using Bag of Words and TF-IDF features, and record its score.
4. Examine where the baseline fails, such as no word meaning, no word order and very large sparse vectors.
5. Train Word2Vec embeddings (CBOW or Skip-gram) on the text, and check which words end up close together.
6. Test words with more than one meaning, such as "bank", to see the limit of one fixed vector per word.

★ PROJECT 1 OF 3

Text Classification, from Word Counts to Sequence Models (contd.)

7. Build an RNN classifier that reads the text word by word, and note its problems with long sentences, slow training and vanishing gradients.
8. Replace it with an LSTM and a GRU, and compare accuracy, training time and parameter count.
9. Compare all four approaches in one results table, and explain which limitation each step was designed to solve.

OUTCOME | **Deliver a text classifier together with a short report on why each approach improved on the one before.**

★ PROJECT 2 OF 3

Multi-Task NLP Toolkit with Transformers and Hugging Face

1. Understand a real-world language problem, such as processing customer feedback, and choose the NLP tasks the toolkit must handle.
2. Use a pretrained Transformer to see how self-attention and positional information handle the "bank" problem that earlier models could not.
3. Choose suitable model types for each task: encoder-only (BERT-style), decoder-only (GPT-style) or encoder-decoder (T5-style).
4. Load models and tokenizers with Hugging Face pipeline and Auto classes, and inspect the config and the tokenization.
5. Build sentiment analysis, named entity recognition and part-of-speech tagging components.
6. Add question answering over a given passage.
7. Add text summarization and translation using sequence-to-sequence models.
8. Generate Hugging Face embeddings and use them for a simple similarity search over the texts.
9. Evaluate the outputs with suitable metrics: accuracy and F1 for classification, BLEU, ROUGE and METEOR for generated text, and perplexity for language models.
10. Extend the toolkit to a non-text input, such as image classification, object detection or audio classification, using Hugging Face.

OUTCOME | **Deliver one toolkit that runs several NLP tasks, with a metrics table for each task.**

★ PROJECT 3 OF 3

LLM-Powered Assistant with Prompt Engineering

1. Understand a real-world task, such as summarizing documents, extracting information or answering questions, and define what a good output looks like.
2. Select two or more LLMs, open and closed, and compare them on cost, quality and context window.
3. Write a baseline prompt with clear system and user instructions, and record the output quality.
4. Tune generation settings such as temperature and top-p, and see their effect on consistency and creativity.
5. Improve the prompt with zero-shot and few-shot prompting.
6. Add chain-of-thought, self-consistency and tree-of-thought prompting for tasks that need reasoning.
7. Test the assistant on tricky inputs to expose hallucinations, outdated knowledge, bias and context window limits.
8. Score each prompt version against a small test set, and keep the best version.

OUTCOME | **Deliver a prompt-driven assistant with a documented prompt history, results and known limitations.**

05

MODULE 5

BUILDING DOCUMENT INTELLIGENCE PIPELINES

LLM Application Engineering with LangChain

1 PROJECT

MODULES / LIBRARIES COVERED DURING THIS MODULE

HuggingFace | Google Gemini API | Groq API | Langchain

1. Introduction to Langchain

- What is document intelligence (extract, summarize, classify) and why LangChain?
- LangChain ecosystem (core, community, provider packages)
- Environment and API key setup
- Import chat models (OpenAI, Gemini, Groq)
- Interacting with chat models: invoke, stream, batch

2. Prompt Templates

- Why templates instead of hard-coded strings
- PromptTemplate and input variables
- ChatPromptTemplate
- System, Human and AI messages (roles and purpose)
- Few-shot prompting with templates (intro)

3. Output Parsers

- Why raw LLM text is hard to use in pipelines
- StrOutputParser
- JsonOutputParser
- Parsing using with_structured_output
- Pydantic parser: define a schema, format instructions, validation
- Handling parsing errors
- Hands-on: extract structured fields (name, date, amount, etc.) into a Pydantic model

4. LCEL Chain

- What is LCEL and the Runnable interface
- The pipe operator: prompt, model, parser
- RunnablePassthrough, RunnableLambda, RunnableParallel

5. Monitoring and Observability

- Introduction to Monitoring and Observability
- Why LLM apps need it (non-deterministic outputs, cost, latency, quality)
- What to track: inputs and outputs, latency, token usage, cost, errors, user feedback
- Observability tools: LangSmith or Langfuse
- Tracing implementation: enable tracing with environment variables
- Tracing implementation: trace chat model calls and add run names, tags and metadata
- Tracing implementation: trace your own Python functions with @traceable
- Reading a trace: runs, nested steps, inputs and outputs, latency and token counts

6. Document Intelligence Pipeline

- Load documents (PDF/text)
- Build a chain for summarization and a chain for structured extraction
- Combine chains with RunnableParallel
- Process a batch of documents and collect results

MODULE PROJECT

★ MODULE PROJECT

Document Intelligence with LangChain

1. Understand a real-world document-handling problem, such as reading invoices, resumes or reports by hand, and define the fields the system must extract, summarize or classify.
2. Set up the LangChain environment with secure API keys, and connect to chat models from more than one provider (OpenAI, Gemini or Groq).
3. Enable tracing with an observability tool such as LangSmith or Langfuse, so every model call is recorded with its inputs, outputs, latency, token usage and errors from the start.
4. Test the models with invoke, stream and batch on sample document text, and compare answers, speed and cost in the traces.
5. Replace hard-coded prompts with reusable prompt templates and chat prompt templates, with clear system, human and AI messages, and add few-shot examples for consistency.
6. Define the output schema with Pydantic, extract fields such as name, date and amount with structured output parsers, and handle parsing errors so bad outputs are caught early.
7. Connect the prompt, model and parser into an LCEL chain, using RunnablePassthrough, RunnableLambda and RunnableParallel to pass inputs along and run steps in parallel.
8. Load documents (PDF and text) and build two chains, one for summarization and one for structured extraction, then combine them so each document is summarized and extracted in a single run.
9. Add run names, tags and metadata to the chains, and trace custom Python functions with @traceable, so each step can be identified in the trace.
10. Process a batch of documents, collect the results into one structured output, and read the traces to inspect each nested step.
11. Use the traces to find slow, costly or failing steps, and improve the prompts, parsers or model choice.

OUTCOME | **Deliver a document intelligence pipeline that turns unstructured documents into validated, structured data and whose behavior and cost can be inspected and improved.**

06

MODULE 6

BUILDING INTELLIGENT RETRIEVAL SYSTEMS: RAG, HYBRID SEARCH & EVALUATION

2 PROJECTS

MODULES / LIBRARIES COVERED DURING THIS MODULE

Langchain | LangGraph | LangSmith | RAGAS

1. RAG Intuition

- What is RAG and why RAG
- RAG vs fine-tuning
- RAG architecture: indexing pipeline and retrieval and generation pipeline
- Common failure points in RAG

2. Document Loading

- Document loaders: PDF, web, CSV, text, directory loaders
- The Document object: page_content and metadata
- Cleaning extracted text (headers, footers, whitespace, noise)
- Challenges: tables, scanned PDFs, multi-column layouts (intro)
- Hands-on: load a mix of PDF, web and CSV sources into a single list of Documents

3. Chunking

- Why chunking matters?
- Chunk size and overlap trade-offs
- Strategies: character, recursive, token-based, and structure-aware (Markdown/HTML headers)
- Semantic chunking (Intuition)

4. Embeddings

- What embeddings are and how they capture meaning
- Similarity measures: cosine, dot product, Euclidean
- Embedding models: OpenAI, Gemini, HuggingFace (open source), and how to choose
- Embedding dimensions, cost and speed

5. Vector Databases and Building the Document Index

- Why vector databases
- FAISS, Chroma, Pinecone, Qdrant, and when to use which
- Building a document index:
- Adding, deleting and avoiding duplicate documents
- Retrievers: as_retriever, top-k, similarity score threshold, MMR

6. Building Metadata

- Why metadata?
- Adding and enriching metadata during loading and chunking
- Metadata filtering at retrieval time
- LLM-generated metadata
- Self-query retriever (intuition)

7. Building the RAG Pipeline with Runnables

- Retrieval chain: retriever, prompt, model, parser
- Formatting retrieved documents into context
- Returning sources and citations with answers
- Conversational RAG: chat history and question rewriting

8. Hybrid RAG

- Keyword search (BM25) vs semantic search, and where each fails
- Hybrid search
- Rank fusion (Reciprocal Rank Fusion) at intuition level
- Re-ranking: cross-encoders and Cohere or HuggingFace rerankers
- Query improvement: query rewriting, multi-query (intuition)

9. Introduction to Evals

- Why evaluating RAG is hard?
- Retrieval metrics vs Generation metrics
- LLM-as-a-judge: concept, rubric design, scoring prompts
- Judging faithfulness, relevance and correctness with structured (Pydantic) outputs

10. RAG Evaluation with RAGAS

- RAGAS overview and the evaluation dataset
- Generating a synthetic test set from documents
- Metrics: faithfulness, answer relevancy, context precision, context recall
- Running RAGAS on the pipeline and reading the results
- Using scores to improve: tune chunk size, k, hybrid search, re-ranking and prompts

MODULE PROJECTS

★ PROJECT 1 OF 2

Document Q&A with RAG

1. Understand a real-world information-access problem and define the questions the system must answer, using RAG instead of fine-tuning.
2. Design the RAG architecture as two pipelines: an indexing pipeline and a retrieval-and-generation pipeline.
3. Load documents from mixed sources (PDF, web pages, CSV and text files) into LangChain Document objects with their metadata.
4. Clean the extracted text by removing headers, footers, extra whitespace and noise, and note problem cases such as tables, scanned PDFs and multi-column layouts.
5. Split the documents into chunks, testing chunk size, overlap and strategies (recursive, token-based and structure-aware) to see their effect on retrieval.
6. Generate embeddings for the chunks with a chosen embedding model, weighing cost, speed and quality.

★ PROJECT 1 OF 2

Document Q&A with RAG (contd.)

7. Build a vector index with a store such as FAISS, Chroma or Qdrant, and handle adding, deleting and duplicate documents.
8. Enrich chunks with metadata such as source, page and section, and filter by it at retrieval time.
9. Build the retrieval chain from retriever, prompt, model and output parser, tuning top-k, score thresholds and MMR.
10. Format retrieved passages into context and return answers with sources and citations.
11. Add chat history and question rewriting so users can ask follow-up questions.
12. Test the system on realistic questions, record where it fails (loading, chunking, retrieval or generation), and fix the main problems.

★ PROJECT 2 OF 2

Advanced RAG with Evaluation

1. Start from the baseline RAG pipeline above and define the quality goals for answers: accurate, grounded and relevant.
2. Identify where keyword search and semantic search each fail, using queries with exact terms and queries with paraphrased wording.
3. Build hybrid retrieval that combines BM25 and vector search, and merge the results with rank fusion.
4. Add re-ranking with a cross-encoder or a hosted reranker to improve the order of retrieved passages.
5. Improve the user's query with rewriting and multi-query retrieval.
6. Create an evaluation dataset by hand and by generating a synthetic test set from the documents.
7. Measure retrieval quality and generation quality separately.
8. Build an LLM-as-a-judge evaluator with a clear rubric and structured (Pydantic) outputs to judge faithfulness, relevance and correctness.
9. Run RAGAS on the pipeline and read faithfulness, answer relevancy, context precision and context recall.
10. Use the scores to tune chunk size, k, hybrid search, re-ranking and prompts, and re-run the evaluation after each change.
11. Compare the baseline and improved pipelines in a before-and-after score table.

OUTCOME | **Deliver a RAG system that is measured and improved with evidence.**

07

MODULE 7

AGENTIC SYSTEMS ENGINEERING WITH LANGGRAPH & MCP

3 PROJECTS

MODULES / LIBRARIES COVERED DURING THIS MODULE

Langchain | LangGraph | LangSmith

1. Agent

- What is an agent?
- The ReAct loop in plain words: Think, Act, Observe
- Hands-on: build an agent with create_agent
- Hands-on: add ready-made tools (Wikipedia, web search)

2. Building Custom Tools

- What a tool is and why the description matters
- The @tool decorator, docstrings and type hints
- Hands-on: build simple tools and return a clean, structured final answer

3. Memory and Handling Errors

- Why an agent forgets, and how a checkpointer fixes it
- Thread_id: separate conversations
- Hands-on: add memory and chat across several turns
- Setting a limit so an agent cannot loop forever

4. Middleware

- What middleware is?
- Hands-on: PII middleware, masking an email or phone number
- Hands-on: tool retry and model retry
- Hands-on: call limits and model fallback

5. Human Approval

- Human-in-the-loop: approve, edit or reject a tool call
- Hands-on: add approval to a sensitive tool
- Build a research assistant with memory, custom tools, PII protection and approval

6. What is MCP?

- The problem MCP solves
- Host, client and server in simple words
- The three things a server offers: tools, resources, prompts
- Two ways to connect: stdio (local) and HTTP (remote)

7. Using MCP Servers in LangChain

- langchain-mcp-adapters and MultiServerMCPClient
- Hands-on: connect to a ready-made server
- Hands-on: load MCP tools and give them to your agent
- Hands-on: use two servers in one agent

8. Build Your First MCP Server

- What FastMCP is and why it is easy
- Hands-on: create a server with @mcp.tool
- Hands-on: call it from your LangChain agent
- Reuse the Week 1 tools as server tools

9. Introduction to LangGraph

- LangChain agent vs LangGraph: when you want more control
- Three ideas: state, nodes, edges
- What is state? A state schema with TypedDict
- Hands-on: build and run a one-node graph

10. Building Workflows

- StateGraph, START, END, compile
- Hands-on: a sequential workflow
- Hands-on: a parallel workflow
- Hands-on: a conditional workflow (route by a condition)
- Draw the graph in code

11. Simple Workflow Patterns

- Prompt chaining (step by step)
- Routing (choose the right path)
- Hands-on: a write, review and revise loop with a retry limit
- Workflow vs agent: a fixed path vs a path the model chooses

12. Chatbot with Memory and Tools

- A chatbot graph that holds a list of messages
- Hands-on: add tools with ToolNode and tools_condition
- Hands-on: add memory with InMemorySaver and thread_id
- Hands-on: stream the chatbot's replies

13. Human in the Loop

- interrupt: pause the graph and wait for a human
- Hands-on: add an approval step before a tool runs
- Optional demo: save memory to a database so it survives a restart

14. Agentic RAG with LangChain

- Quick recap of RAG
- Classic RAG vs agentic RAG (the agent decides when to look things up)
- Hands-on: turn a retriever into a tool
- Hands-on: build an agent with two knowledge sources as two tools

15. Agentic RAG with LangGraph

- The idea: retrieve, check if the documents are useful, rewrite the question if not, then answer
- Hands-on: a graph with retrieve, grade and generate steps
- Hands-on: add "rewrite the question and try once more" with a limit

16. Agent Evals: Seeing What Your Agent Does

- Why testing agents is different?
- What to check: the final answer, which tools were used, speed and cost
- Hands-on: turn on LangSmith tracing and read a trace

17. Agent Evals: Writing Evaluators

- Hands-on: an LLM-as-a-judge evaluator with a short rubric
- Hands-on: a tool check (did the agent call the expected tool?)
- Read failed traces and find the cause (retrieval, wrong tool, or prompt)
- Hands-on: make one change, such as a better tool description or prompt, and run the evals again

18. Multi-Agent Basics

- Why use many agents?
- When not a single agent is enough?
- The supervisor idea: a manager agent and worker agents
- Hands-on: a supervisor agent that calls a researcher agent and a writer agent as tools

19. Supervisor with LangGraph

- The supervisor graph: the supervisor picks a worker, the worker reports back
- Hands-on: build it in LangGraph with a researcher and a writer
- Hands-on: trace a run and see the handoffs
- Mention the prebuilt supervisor helper after building by hand

20. Deployment and Agent Chat UI

- A LangGraph project layout and langgraph.json
- Hands-on: run the app locally with the LangGraph dev server
- Hands-on: connect Agent Chat UI and chat with your agent

MODULE PROJECTS

★ PROJECT 1 OF 3

Tool-Using AI Agents

1. Understand a real-world business problem and define the objectives and success criteria for an AI agent.
2. Design the agent's workflow using the ReAct (Think, Act, Observe) loop, and build it with LangChain.
3. Equip the agent with ready-made and custom tools, with clear descriptions and structured outputs.

★ PROJECT 1 OF 3

Tool-Using AI Agents (contd.)

4. Add memory, so the agent keeps context across turns, with a separate conversation for each user.
5. Make the agent reliable by setting call limits and adding middleware for PII masking, retries and model fallback.
6. Add human-in-the-loop approval for sensitive actions.
7. Expose reusable tools through an MCP server built with FastMCP, and connect the agent to it along with existing MCP servers.
8. Test the agent on realistic tasks, review each step it takes, and refine its tools and prompts.

OUTCOME | **Build an end-to-end AI assistant that can search, reason and act safely on real-world tasks.**

★ PROJECT 2 OF 3

Agentic RAG with Evaluation

1. Understand a real-world information-access problem and define the questions the system must answer.
2. Convert one or more knowledge sources into retriever tools that an agent can call.
3. Build an agent that decides when to retrieve, and compare it with classic RAG.
4. Rebuild the system as a LangGraph workflow with state, nodes and edges, including memory and streaming responses.
5. Add a corrective flow that retrieves documents, grades them, rewrites the question when results are weak, and answers within a retry limit.
6. Add a human approval step before sensitive actions.
7. Trace the application with LangSmith and inspect the tools and documents behind each answer.
8. Create a test set and evaluate the system with LLM-as-a-judge and tool-call evaluators.
9. Analyze failures, apply targeted improvements, and compare before-and-after scores.

OUTCOME | **Build a reliable knowledge assistant that is measured, not just demonstrated.**

★ PROJECT 3 OF 3

Multi-Agent Systems and Deployment

1. Understand a complex business task and divide it into specialist roles.
2. Build worker agents for each role, with their own tools and prompts.
3. Add a supervisor agent that delegates to workers and combines their results, starting with the simplest version.
4. Rebuild the system as a LangGraph supervisor graph with clear handoffs between workers.
5. Add a draft, review and revise loop with a retry limit and a human approval step.

★ PROJECT 3 OF 3

Multi-Agent Systems and Deployment (contd.)

6. Trace runs to follow handoffs, tool calls and token use, and score output quality with a rubric.
7. Organize the application as a LangGraph project with langgraph.json and run it with the LangGraph dev server.
8. Connect Agent Chat UI so users can interact with the system through a chat interface.

OUTCOME | **Build a deployable multi-agent system for real-world business tasks.**

INNOMATICS®
RESEARCH LABS

THANK YOU

Thank you for your time and interest in Innomatics Research Labs.

FOLLOW US ON

Instagram
Facebook
Linkedin
Youtube
Website

www.innomatics.in

+91 99516 66670/71/72/74